



EC200U&EG91xU Series QuecOpen

Secure Boot User Guide

LTE Standard Module Series

Version: 1.1

Date: 2023-08-21

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-12-22	Neo KONG	Creation of the document
1.0	2022-03-08	Neo KONG	First official release
1.1	2023-08-21	Arvin XU	Added applicable modules EG912U-GL and EG915U series.

Contents

About the Document.....	3
Contents	4
Table Index.....	5
Figure Index	6
1 Introduction	7
1.1. Applicable Modules	7
2 Secure Boot Overview.....	8
2.1. Definition	8
2.2. Digital Signature	8
3 Introduction to Secure Boot Solution.....	9
3.1. Generate Keys	9
3.2. Sign Images	9
3.3. Enable Secure Boot	10
3.4. Signature Verification Process.....	10
3.4.1. Secure Boot Not Enabled	10
3.4.2. Secure Boot Enabled	10
3.5. Trust Chain.....	11
3.5.1. Trust Chain for Downloading (Multilevel Verification)	11
3.5.2. Trust Chain for Booting (Multilevel Verification).....	11
4 Enable Secure Boot.....	12
4.1. Generate Keys	12
4.2. Update Keys.....	13
4.3. Turn On Secure Boot Macro	15
4.4. Enable Secure Boot Enabling Function	15
4.5. Compile Firmware Package.....	16
4.6. Download Firmware Package	17
5 Appendix References	18

Table Index

Table 1: Applicable Modules.....	7
Table 2: Related Document.....	18
Table 3: Terms and Abbreviations	18

Figure Index

Figure 1: Key Storage Location	9
Figure 2: Trust Chain for Downloading	11
Figure 3: Trust Chain for Booting	11
Figure 4: Generate New Keys.....	13
Figure 5: Reconfigure Keys in CSDK.....	14
Figure 6: Reconfigure Keys in APP SDK	14
Figure 7: Turn On Secure Boot Macro	15
Figure 8: Uncomment Secure Boot Enabling Function	15
Figure 9: Files Generated After Compiling CSDK.....	16
Figure 10: Files Generated After Compiling APP SDK	16
Figure 11: Successful Verification	17
Figure 12: Failed Verification.....	17

1 Introduction

Quectel LTE Standard EC200U series and EG91xU family modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document explains the Secure Boot function of Quectel EC200U series and EG91xU family modules in QuecOpen® solution, and steps for enabling it.

NOTE

The SDKs provided by Quectel contain APP SDK and CSDK. APP SDK contains Kernel, firmware generated after compilation and sample source codes. CSDK contains Kernel, firmware generated after compilation, sample source codes and some source files of chip vendor.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Model
-	EC200U Series
EG91xU	EG912U-GL
	EG915U Series

2 Secure Boot Overview

2.1. Definition

Secure Boot refers to the boot-up sequence established on a trusted platform. The Secure Boot system adds signature verification to each stage of the module's downloading and booting process to prevent any unauthorized or maliciously modified software from running on the module. A root of trust is needed during the downloading and booting process. Bootrom is embedded in EC200U series and EG91xU family QuecOpen modules as a firmware which is unmodifiable and therefore can serve as the root of trust.

Secure Boot is intended to:

1. Prohibit the downloading of unauthorized firmware.
2. Prohibit the running of unauthorized firmware.
3. Prohibit the illegal tracking and debugging of code.
4. Allow secure upgrade.

2.2. Digital Signature

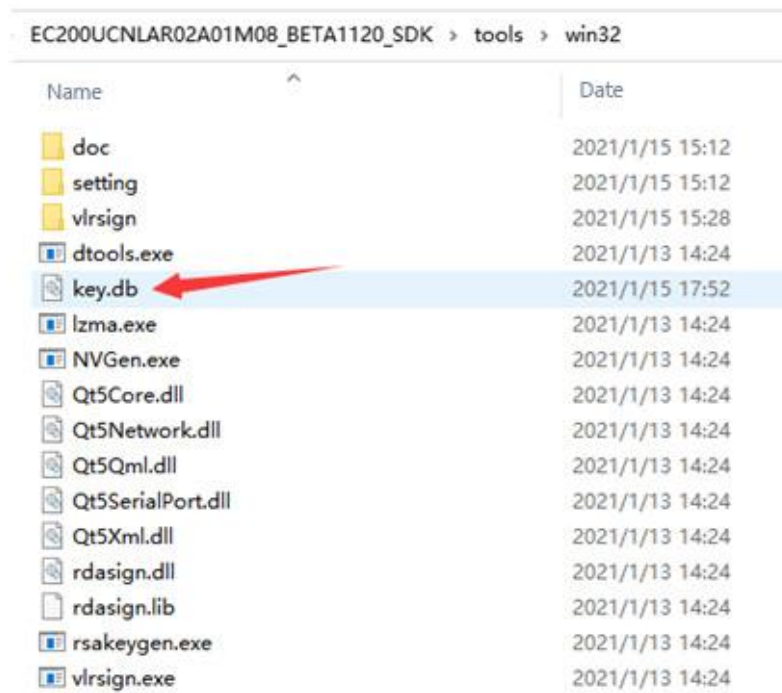
The digital signature technology used by the module is based on asymmetric encryption algorithm. Asymmetric encryption algorithm divides keys into public key and private key. The private key encrypts certain information, and the public key verifies the decryption information. The contents encrypted with the public key can be decrypted by private key, and the contents signed with the private key can be authenticated by public key. If you only have the private key or public key, you are unable to get the other key.

It takes a long time to sign images with the digital signature algorithm. To improve efficiency, the private key is used to encrypt the Hash value of the image. The cipher text is called the signature and it is attached to the images. The recipient uses the public key to decrypt the Hash value of the image and get the decrypted Hash value, and then recalculate the image to get another Hash value. If the two Hash values are identical, the signature is verified and then the firmware is downloaded or the program is started to continue the next process. If the two Hash values are different, the signature verification fails, and firmware downloading or module startup exits.

3 Introduction to Secure Boot Solution

3.1. Generate Keys

The module generates the keys (public Key and private Key) with `rsakeygen.exe`, which is in the `tools\win32` directory in SDK. The generated keys are stored in `key.db` file under the `tools\win32` directory. The private key must be kept safe and cannot be disclosed.



Name	Date
doc	2021/1/15 15:12
setting	2021/1/15 15:12
vlrsign	2021/1/15 15:28
dtools.exe	2021/1/13 14:24
key.db	2021/1/15 17:52
lzma.exe	2021/1/13 14:24
NVGen.exe	2021/1/13 14:24
Qt5Core.dll	2021/1/13 14:24
Qt5Network.dll	2021/1/13 14:24
Qt5Qml.dll	2021/1/13 14:24
Qt5SerialPort.dll	2021/1/13 14:24
Qt5Xml.dll	2021/1/13 14:24
rdasign.dll	2021/1/13 14:24
rdasign.lib	2021/1/13 14:24
rsakeygen.exe	2021/1/13 14:24
vlrsign.exe	2021/1/13 14:24

Figure 1: Key Storage Location

3.2. Sign Images

The module signs the images with `vlrsign.exe`, which is in the `tools\win32` directory in SDK. `vlrsign.exe` can only be executed from the command line. You can refer to `vlrsign.md` under the `tools\win32\doc` directory for the use of `vlrsign.exe`.

The signing process:

1. vlrsign.exe uses blake2 hash algorithm to calculate the images to be signed to generate the Hash value. The images are generated after compilation, among which *fdL1*, *fdl2*, *boot*, *ApSystemProgram* and *appimage* need to be signed.
2. vlrsign.exe uses the private key saved in *key.db* to sign the Hash value generated in **step 1** (using ed25519 digital signature algorithm), and then attaches the signature to the corresponding image.
3. vlrsign.exe attaches the public key to *boot.img* and downloads the public key with the firmware to the flash of the module.

3.3. Enable Secure Boot

You need to change the status of `QL_APP_FEATURE_SECURE_BOOT` in `components\ql-application\ql_app_feature_config.cmake` of SDK to ON and then reboot the module. After the module is rebooted, the system automatically writes the public key attached to *boot.img* to efuse to enable Secure Boot function. The public key cannot be changed after it is written to the efuse and the enabling flag of Secure Boot in efuse is set. See **Chapter 4** for detailed information.

3.4. Signature Verification Process

3.4.1. Secure Boot Not Enabled

If Secure Boot is not enabled, the system skips the signature verification process and downloads firmware or starts up directly.

3.4.2. Secure Boot Enabled

If Secure Boot is enabled, the module restarts and enters the signature verification process.

1. The system uses the public key in efuse to decrypt the signature attached to *boot.img* to obtain the Hash value.
2. Compare the new Hash value with the original Hash value. If the two Hash values are identical, the verification is passed and the module downloads firmware or starts up normally. If the two Hash values are different, the verification fails and the module cannot download firmware or start up.

3.5. Trust Chain

3.5.1. Trust Chain for Downloading (Multilevel Verification)

As the root of trust, Bootrom is the firmware embedded in chips and cannot be modified. Therefore, the Bootrom is trustworthy and used to perform authentication in the next booting stage. Bootrom downloads *fdl1* through UART or USB to RAM and verifies *fdl1*. After the verification is passed, *fdl1* can download and verify *fdl2*. Once *fdl2* is verified, *fdl2* downloads the *BootLoader*, *ApSystemProgram* and *APP* to the flash of module and verifies them.

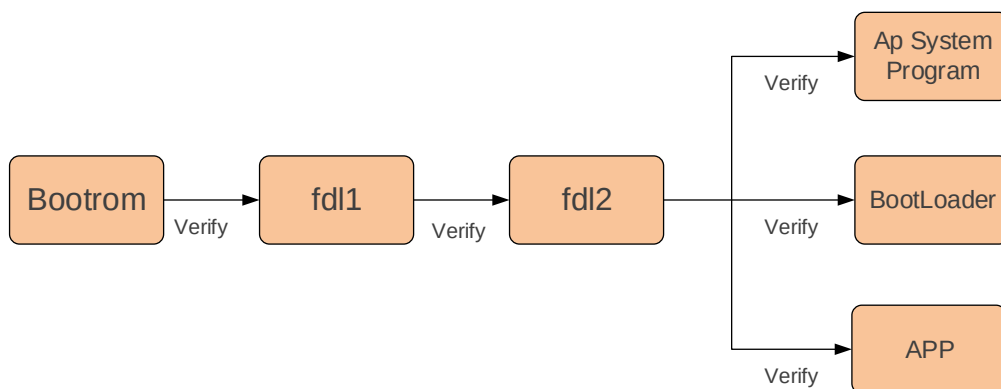


Figure 2: Trust Chain for Downloading

3.5.2. Trust Chain for Booting (Multilevel Verification)

The boot-up process of the module is a multi-level loading process, i.e., the code of the previous level loads the code of next level. When Secure Boot is enabled, the code of each level is verified before being loaded. If the verification fails, the module fails to boot up.

Bootrom is trustworthy (the root trusted entity in booting process) and used for authentication in the next booting stage. The next booting stage is BootLoader, which is executed after BootLoader signature is verified successfully. BootLoader verifies Ap System Program signature when loading Ap System Program, and ApSystemProgram runs once the verification is passed. Ap System Program verifies the APP signature when loading APP, and APP runs once the verification is passed. This is the trust chain for the entire firmware boot-up process.

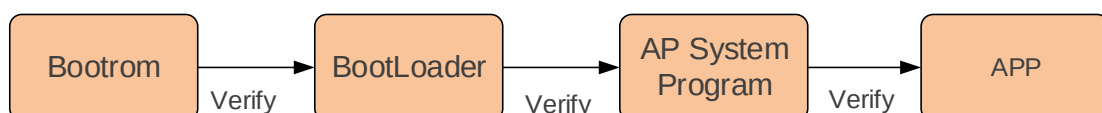


Figure 3: Trust Chain for Booting

4 Enable Secure Boot

The Secure Boot enabling operation has been encapsulated in the software of Quectel EC200U series and EG91xU family QuecOpen module, and the steps are as follows.

- Step 1:** Run `rsakeygen.exe`, enter the password and product name to generate keys;
- Step 2:** Replace the value of `sign_password` in `chip_8910.cmake` (CSDK) or `CMakeLists.txt` (APP SDK) with the password entered when running `rsakeygen.exe`; Replace the value of `sign_product` in `chip_8910.cmake` (CSDK) or `CMakeLists.txt` (APP SDK) with the product name entered when running `rsakeygen.exe`;
- Step 3:** Change the status of `QL_APP_FEATURE_SECURE_BOOT` in `ql_app_feature_config.cmake` to ON;
- Step 4:** Uncomment `ql_dev_enable_secure_boot()` in `ql_init.c`;
- Step 5:** Run `buildxxx.bat` to compile the source codes to generate a firmware. The generated firmware carries the signature signed with the new keys and enables Secure Boot. After the firmware is downloaded to the module, the Secure Boot function is enabled.

NOTE

The SDKs provided by Quectel contain APP SDK and CSDK. APP SDK contains Kernel, firmware generated after compilation and sample source codes. CSDK contains Kernel, firmware generated after compilation, sample source codes and some source files of chip vendor.

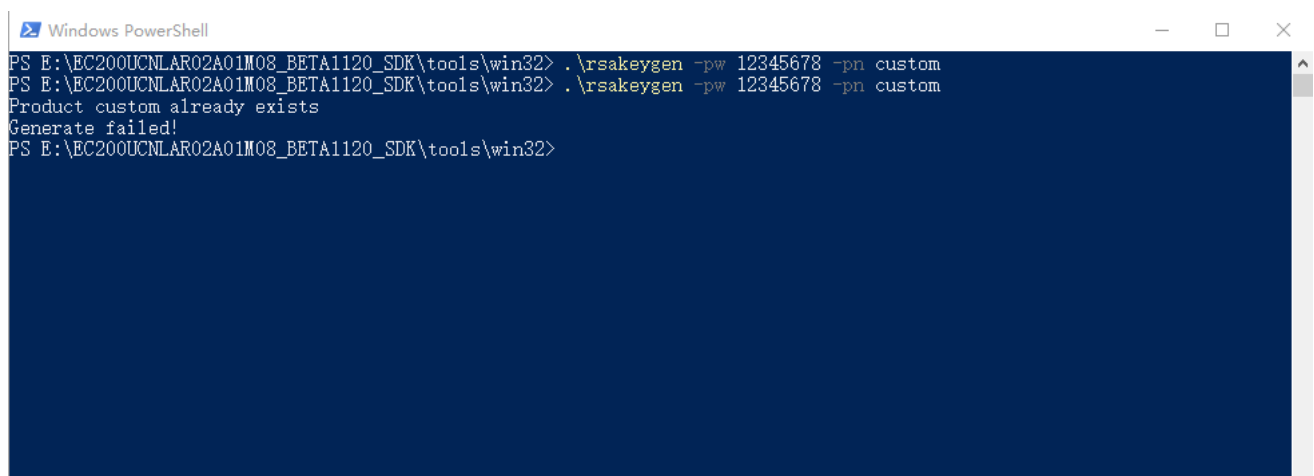
4.1. Generate Keys

You need to run `rsakeygen.exe` to generate the keys. For the use of `vlrsign.exe`, see `vlrsign.md` under the directory of `tools\win32\doc`. `rsakeygen.exe` can only be executed from the command line. Command line options:

- v, --version** Displays the version information.
- pw** Enters the password, which is a 8-bit string of ASCII characters. `vlrsign` verifies the password based on the read signing key.
- pn** Enters the product name, which is a string of ASCII characters no more than 49 bits. `vlrsign` retrieves the corresponding signing key according to the entered product name.

Even if the input parameters are the same, the public and private key pairs generated by `rsakeygen.exe` each time are different, and the generated `key.db` is also different. If you execute the command repeatedly to generate keys, the public and private key pair generated last time is not overwritten. The public and private key pairs generated by `rsakeygen.exe` are encrypted and stored in `key.db` under the `tools\win32`. `key.db` directory that contains sensitive information, so it must be kept secure and cannot be disclosed or lost.

If `key.db` is leaked, others can use the `key.db` to sign the firmware. The public key in `key.db` is the same as the public key in the efuse of terminal, which can be verified by Secure Boot. If `key.db` is lost, `rsakeygen.exe` cannot regenerate a key pair of which the public key is the same as the public key in the efuse of terminal. In this case, subsequent firmware cannot be signed, and the signed firmware cannot be upgraded.



```

Windows PowerShell
PS E:\EC200UCNLR02A01M08_BETA1120_SDK\tools\win32> .\rsakeygen -pw 12345678 -pn custom
PS E:\EC200UCNLR02A01M08_BETA1120_SDK\tools\win32> .\rsakeygen -pw 12345678 -pn custom
Product custom already exists
Generate failed!
PS E:\EC200UCNLR02A01M08_BETA1120_SDK\tools\win32>
  
```

Figure 4: Generate New Keys

4.2. Update Keys

The image signing process of Quectel EC200U series and EG91xU family QuecOpen module has been integrated into the integrated compilation of Windows and Linux platforms. The images of `fdl1`, `fdl2`, `boot`, `ApSystemProgram` and `appimage` are automatically signed after compilation and form a package with the suffix `.pac`. The test keys are used by default, i.e., Product Name is `test` and Password is `12345678`. In actual operations, you need to replace the default test keys with the keys generated in the manner referred to in **Chapter 4.1**.

- CSDK

After the public key and private key are generated, you need to replace the value of `sign_password` in `components\chip\chip_8910.cmake` of SDK with the password entered when running `rsakeygen.exe` and replace the value of `sign_product` in `components\chip\chip_8910.cmake` of SDK with the product name

entered when running rsakeygen.exe.

```

set(unittest_ldscript ${SOURCE_TOP_DIR}/components/hal/ldscripts/flashrun.ld)
set(pac_fdl_files ${out_hex_dir}/fdll.sign.img ${out_hex_dir}/fdl2.sign.img)
set(sign_password 12345678) # customer product shall replace with customer's key
set(sign_product test) # customer product shall replace with customer's product name

function(sign_image src dst)
    add_custom_command(OUTPUT ${dst}
        COMMAND vlrsign --pw ${sign_password} --pn ${sign_product} --ha Blake2
        --img ${src} --out ${dst}
        DEPENDS ${src}
    )
endfunction()

function(pacvariant_gen)
    # Sign fdl1/fdl2/boot/ap. They are common to all variants
    set(sign_boot_padlen 0xbce0) # can't be changed
    add_custom_command(OUTPUT ${pac_fdl_files} ${out_hex_dir}/boot.sign.img ${out_hex_dir}/${BUILD_TARGET}.sign.img ${out_hex_dir}/${BUILD_TARGET}.boot.sign.img
        COMMAND vlrsign --pw ${sign_password} --pn ${sign_product} --ha Blake2
        --img ${out_hex_dir}/boot.img --out ${out_hex_dir}/boot.sign.img --plen ${sign_boot_padlen}
        COMMAND vlrsign --pw ${sign_password} --pn ${sign_product} --ha Blake2
        --img ${out_hex_dir}/fdll.img --out ${out_hex_dir}/fdll.sign.img
    )
endfunction()

```

Figure 5: Reconfigure Keys in CSDK

● APP SDK

After the public key and private key are generated, you need to replace the value of *sign_password* in *CMakeLists.txt* under the root directory of APP SDK with the password entered when running rsakeygen.exe and replace the value of *sign_product* in *CMakeLists.txt* under the root directory of APP SDK with the product name entered when running rsakeygen.exe.

```

if(CONFIG_QUEC_PROJECT_FEATURE_SECURE_BOOT)
    if(QL_APP_FEATURE_SECURE_BOOT)
        # Sign fdl1/fdl2/boot/ap. They are common to all variants
        set(img_files ${SOURCE_TOP_DIR}/components/lib)
        set(sign_password 12345678) # customer product shall replace with customer's key
        set(sign_product test) # customer product shall replace with customer's product name
        set(sign_boot_padlen 0xbce0) # can't be changed
        execute_process(
            COMMAND ${SOURCE_TOP_DIR}/tools/win32/vlrsign/VLRSign --pw ${sign_password} --pn ${sign_product} --ha Blake2
            --img ${img_files}/8915DM_cat1_open.img --out ${img_files}/8915DM_cat1_open.sign.img
            COMMAND ${SOURCE_TOP_DIR}/tools/win32/vlrsign/VLRSign --pw ${sign_password} --pn ${sign_product} --ha Blake2
            --img ${img_files}/boot.img --out ${img_files}/boot.sign.img --plen ${sign_boot_padlen}
            COMMAND ${SOURCE_TOP_DIR}/tools/win32/vlrsign/VLRSign --pw ${sign_password} --pn ${sign_product} --ha Blake2
            --img ${img_files}/fdll.img --out ${img_files}/fdll.sign.img
            COMMAND ${SOURCE_TOP_DIR}/tools/win32/vlrsign/VLRSign --pw ${sign_password} --pn ${sign_product} --ha Blake2
            --img ${img_files}/fdl2.img --out ${img_files}/fdl2.sign.img
            DEPENDS ${img_files}/8915DM_cat1_open.img ${img_files}/boot.img ${img_files}/fdll.img ${out_hex_dir}/appimage.img
            ${img_files}/fdl2.img
        )
    endif()
endif()

```

Figure 6: Reconfigure Keys in APP SDK

4.3. Turn On Secure Boot Macro

You can open *ql_app_feature_config.cmake* under the *components\ql-application* directory and then change the status of `QL_APP_FEATURE_SECURE_BOOT` from OFF to ON.

```

if(CONFIG_QUEC_PROJECT_FEATURE_SECURE_BOOT)
    option(QL_APP_FEATURE_SECURE_BOOT "Enable SECURE BOOT" OFF)
else()
    message(STATUS "FEATURE SECURE BOOT is disabled at core!")
    option(QL_APP_FEATURE_SECURE_BOOT "Enable SECURE BOOT" OFF)
endif()
message(STATUS "QL_APP_FEATURE_SECURE_BOOT ${QL_APP_FEATURE_SECURE_BOOT}")
    
```

Figure 7: Turn On Secure Boot Macro

4.4. Enable Secure Boot Enabling Function

You can enable (uncomment) the Secure Boot enabling function in *ql_init.c* under the *components\ql-application\init* directory.

```

static void ql_init_demo_thread(void *param)
{
    QL_INIT_LOG("init demo thread enter, param 0x%x", param);
    #ifdef QL_APP_FEATURE_SECURE_BOOT
        //ql_dev_enable_secure_boot();
    #endif

    #if 0
        ql_gpio_app_init();
        ql_gpioint_app_init();
    #endif

    #ifdef QL_APP_FEATURE_LEDCFG
        ql_ledcfg_app_init();
    #endif

    #ifdef QL_APP_FEATURE_AUDIO
        //ql_audio_app_init();
    #endif
}
    
```

Figure 8: Uncomment Secure Boot Enabling Function

4.5. Compile Firmware Package

You can use the *build_all.bat* (CSDK) or *build_app.bat* (APP SDK) under the root directory to compile the CSDK or APP SDK to generate a firmware. The generated firmware carries the signature signed with the new keys and enables Secure Boot. Once the compilation is completed, a folder named *traget* is generated under the root directory of SDK to store the firmware package.

The files output after compiling CSDK are listed below. *8915DM_cat1_open_EC200UCNLAR01A01M08.pac* only contains Kernel and *8915DM_cat1_open_EC200UCNLAR01A01M08_merge.pac* contains Kernel and appimage.

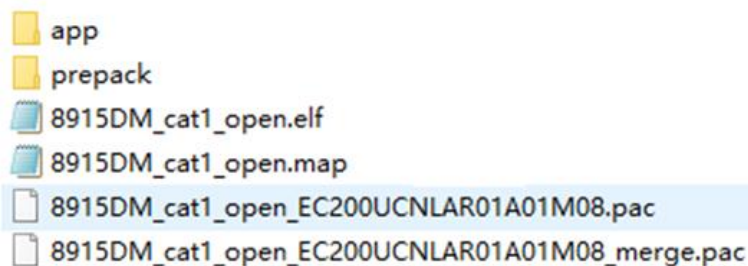


Figure 9: Files Generated After Compiling CSDK

The files generated after compiling APP SDK are listed below. *appimage.pac* only contains appimage and *8915DM_cat1_open_core.pac* contains Kernel and appimage.

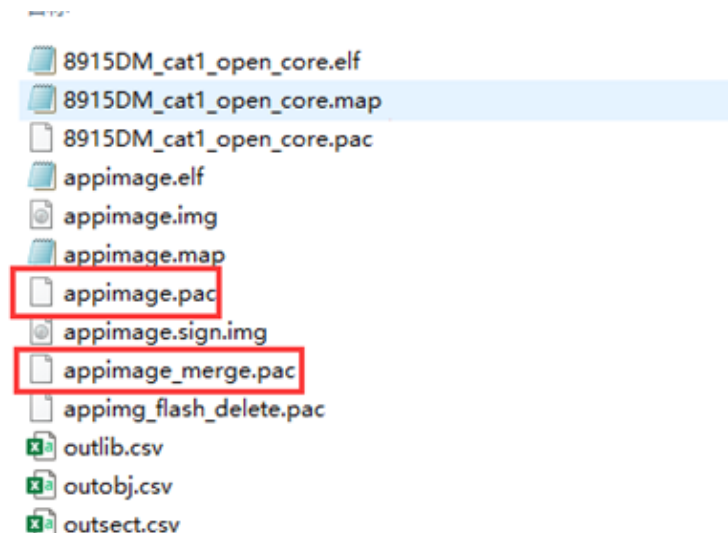
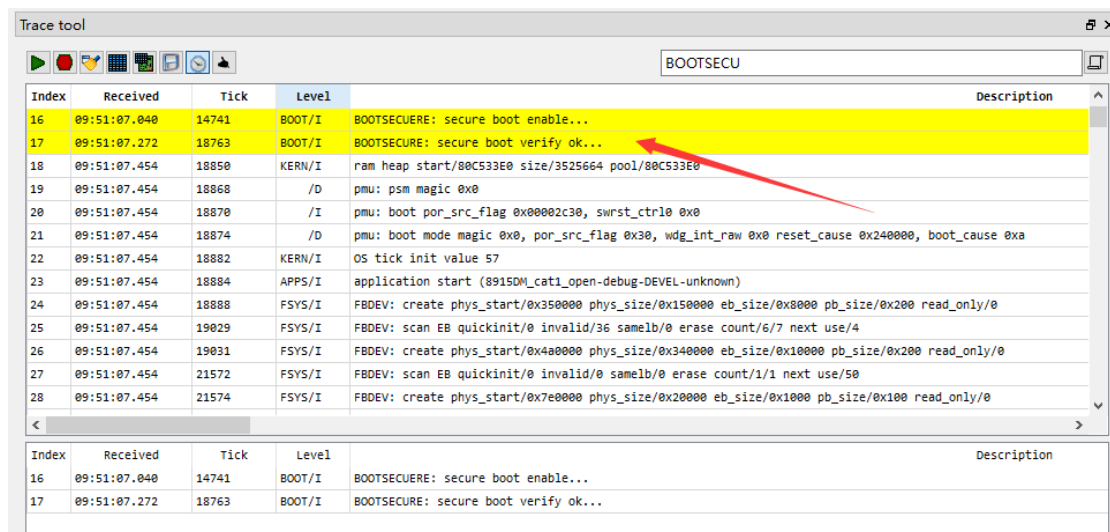


Figure 10: Files Generated After Compiling APP SDK

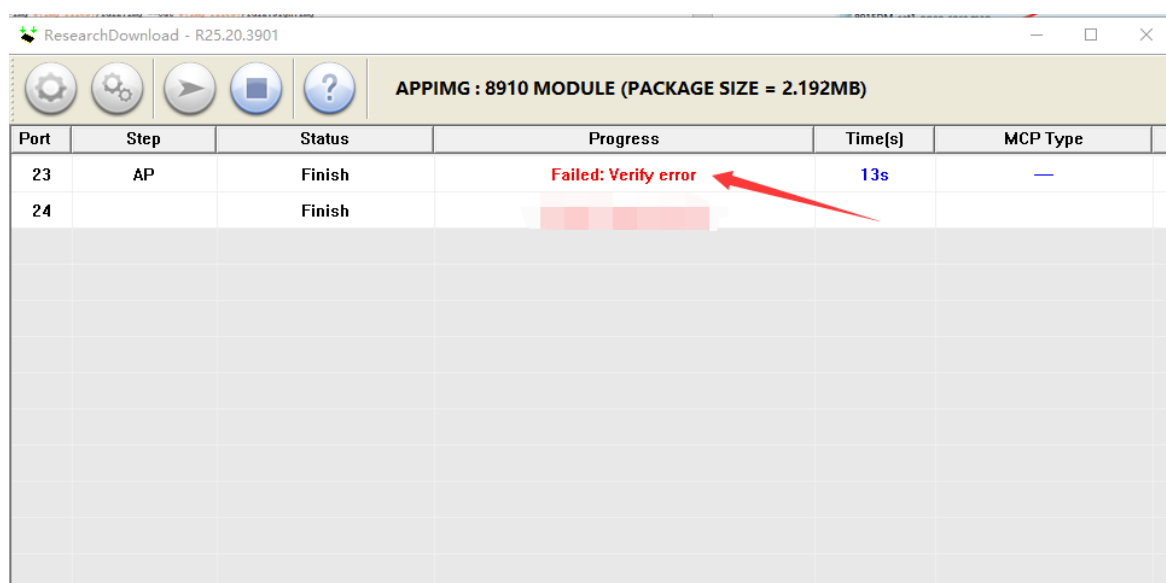
4.6. Download Firmware Package

Once the firmware generated after compilation is downloaded to the module according to the used SDK and the actual needs, you can reboot the module to enable the Secure Boot function. The keys are written to the efuse and cannot be modified. Signature verification is performed in the subsequent downloading and booting process of the module. If the verification is successful, the module can normally download firmware or start up; if the verification fails, the module fails to download firmware or start up. That ensures that the user's code, instead of the software that has not been legally signed or maliciously modified software, is running on the module.



Index	Received	Tick	Level	Description
16	09:51:07.040	14741	BOOT/I	BOOTSECURE: secure boot enable...
17	09:51:07.272	18763	BOOT/I	BOOTSECURE: secure boot verify ok...
18	09:51:07.454	18850	KERN/I	ram heap start/00C533E0 size/3525664 pool/00C533E0
19	09:51:07.454	18868	/D	pmu: psm magic 0x0
20	09:51:07.454	18870	/I	pmu: boot por_src_flag 0x00002c30, swrst_ctrl0 0x0
21	09:51:07.454	18874	/D	pmu: boot mode magic 0x0, por_src_flag 0x30, wdg_int_raw 0x0 reset_cause 0x240000, boot_cause 0xa
22	09:51:07.454	18882	KERN/I	OS tick init value 57
23	09:51:07.454	18884	APPS/I	application start (8915DM_cat1_open-debug-DEVEL-unknown)
24	09:51:07.454	18888	FSYS/I	FBDEV: create phys_start/0x350000 phys_size/0x150000 eb_size/0x8000 pb_size/0x200 read_only/0
25	09:51:07.454	19029	FSYS/I	FBDEV: scan EB quickinit/0 invalid/36 same1b/0 erase count/6/7 next use/4
26	09:51:07.454	19031	FSYS/I	FBDEV: create phys_start/0x4a0000 phys_size/0x340000 eb_size/0x10000 pb_size/0x200 read_only/0
27	09:51:07.454	21572	FSYS/I	FBDEV: scan EB quickinit/0 invalid/0 same1b/0 erase count/1/1 next use/50
28	09:51:07.454	21574	FSYS/I	FBDEV: create phys_start/0x7e0000 phys_size/0x20000 eb_size/0x1000 pb_size/0x100 read_only/0

Figure 11: Successful Verification



Port	Step	Status	Progress	Time[s]	MCP Type
23	AP	Finish	Failed: Verify error	13s	—
24		Finish			

Figure 12: Failed Verification

5 Appendix References

Table 2: Related Document

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen_CSDK_Quick_Start_Guide

Table 3: Terms and Abbreviations

Abbreviation	Description
APP	Application
ASCII	American Standard Code for Information Interchange
IoT	Internet of Things
LTE	Long-Term Evolution
RAM	Random Access Memory
RTOS	Real-Time Operating System
SDK	Software Development Kit
UART	Universal Asynchronous Receiver/Transmitter
USB	Universal Serial Bus